UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
Departamento de Computação - DC
Graduação em Ciência da Computação
Disciplina: Sistemas Distribuídos
Prof.: Paulo Henrique Lopes Silva

Prática 3.0.1 – Cliente/Servidor TCP Básico
Mensagens em Bytes

1. Servidor.

Java

```java
public class Servidor {
        public static void main(String[] args) {
        int porta = 12345; // Define a porta do servidor

           try {

                ServerSocket servidorSocket = new ServerSocket(porta);
                System.out.println("Servidor aguardando conexões na porta "
                        + porta
                        + "...");

                // Aguarda conexão do cliente
                Socket socketCliente = servidorSocket.accept();
                System.out.println("Cliente conectado!");

                // Fluxo de entrada para receber dados
                /* A classe InputStream é a classe base para todas as entradas de dados
                binários em Java.
                Ela permite ler bytes a partir de arquivos, conexões de rede, memória, etc..
                */

                InputStream fluxoEntrada = socketCliente.getInputStream();
                byte[] buffer = new byte[1024]; // Buffer para leitura
                // Lê os dados enviados pelo cliente
                int bytesLidos = fluxoEntrada.read(buffer);

                // Converte bytes para string
                String mensagemRecebida = new String(buffer, 0, bytesLidos);
                System.out.println("Mensagem recebida: " + mensagemRecebida);

                // Fluxo de saída para enviar resposta
                /* A classe OutputStream é a classe base para todas as saídas de dados
                binários em Java.
                Ela permite gravar bytes em arquivos, conexões de rede, memória, etc.. */

                OutputStream fluxoSaida = socketCliente.getOutputStream();
                String resposta = "Mensagem recebida com sucesso!";
                fluxoSaida.write(resposta.getBytes()); // Envia resposta em bytes

                // Fecha fluxos e socket
                fluxoEntrada.close();
                fluxoSaida.close();
                socketCliente.close();
                System.out.println("Conexão encerrada.");

           } catch (IOException e) {
                System.err.println("Erro no servidor: " + e.getMessage());
           }
        }
}
```

2. Cliente.

```java
public class Cliente {
        public static void main(String[] args) {
        String host = "localhost"; // Endereço do servidor
        int porta = 12345; // Porta do servidor

           try {
                Socket socketCliente = new Socket(host, porta);
                System.out.println("Conectado ao servidor!");

                // Scanner para entrada do usuário
                Scanner sc = new Scanner(System.in);
                System.out.print("Digite uma mensagem para o servidor: ");
                String mensagem = sc.nextLine();

                // Fluxo de saída para enviar dados
                OutputStream fluxoSaida = socketCliente.getOutputStream();
                // Converte string para bytes e envia
                fluxoSaida.write(mensagem.getBytes());

                // Fluxo de entrada para receber resposta
                InputStream fluxoEntrada = socketCliente.getInputStream();
                byte[] buffer = new byte[1024];
                // Lê resposta do servidor
                int bytesLidos = fluxoEntrada.read(buffer);

                // Converte bytes recebidos em string
                String resposta = new String(buffer, 0, bytesLidos);
                System.out.println("Resposta do servidor: " + resposta);

                // Fecha fluxos e socket
                fluxoSaida.close();
                fluxoEntrada.close();
                socketCliente.close();
                sc.close();

           } catch (IOException e) {
                System.err.println("Erro no cliente: " + e.getMessage());
           }
        }
}
```

3. Cliente/Servidor.

- Fluxo:
    - Cliente conecta e manda "oi, servidor".
    - Servidor imprime e responde "oi, cliente".
    - Cliente recebe, imprime e fecha.

```java
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.ServerSocket;
import java.net.Socket;
```

```java
public class ImplServidor {

    private int porta;

    public ImplServidor(int porta) {
        this.porta = porta;
        rodar();
    }

    private void rodar() {
        ServerSocket servidor = null;
        try {
            servidor = new ServerSocket(porta);
            System.out.println("Servidor aguardando conexões na porta " +
                      porta + "...");

            while (true) {
                Socket cliente = servidor.accept();
                System.out.println("Cliente conectado: " +
                       cliente.getInetAddress());

                try {
                    InputStream entrada = cliente.getInputStream();
                    OutputStream saida = cliente.getOutputStream();

                    byte[] buffer = new byte[256];
                    int bytesLidos = entrada.read(buffer);

                    if (bytesLidos > 0) {
                        String msgRecebida =
                                new String(buffer, 0, bytesLidos, "UTF-8");
                        System.out.println("Mensagem recebida: " + msgRecebida);

                        String resposta = "oi, cliente";
                        saida.write(resposta.getBytes("UTF-8"));
                        System.out.println("Resposta enviada: " + resposta);
                    }

                    entrada.close();
                    saida.close();
                    cliente.close();

                } catch (IOException e) {
                    System.err.println("Erro na comunicação com cliente: " +
                              e.getMessage());
                }
            }

        } catch (IOException e) {
            System.err.println("Erro no servidor: " + e.getMessage());
        } finally {
            if (servidor != null) {
                try {
                    servidor.close();
                } catch (IOException e) {
                    System.err.println("Erro ao fechar servidor: " +
                              e.getMessage());
                }
            }
        }
    }

    public static void main(String[] args) {
        new ImplServidor(12345);
```

```
    }
}
```

Java

```java
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;

public class ImplCliente {

    private String host;
    private int porta;

    public ImplCliente(String host, int porta) {
        this.host = host;
        this.porta = porta;
        rodar();
    }

    private void rodar() {
        Socket socket = null;
        try {
            socket = new Socket(host, porta);
            System.out.println("Conectado ao servidor!");

            OutputStream saida = socket.getOutputStream();
            InputStream entrada = socket.getInputStream();

            String mensagem = "oi, servidor";
            saida.write(mensagem.getBytes("UTF-8"));
            System.out.println("Mensagem enviada: " + mensagem);

            byte[] buffer = new byte[256];
            int bytesLidos = entrada.read(buffer);

            if (bytesLidos > 0) {
                String resposta = new String(buffer, 0, bytesLidos, "UTF-8");
                System.out.println("Resposta recebida: " + resposta);
            }

            entrada.close();
            saida.close();

        } catch (IOException e) {
            System.err.println("Erro no cliente: " + e.getMessage());
        } finally {
            if (socket != null) {
                try {
                    socket.close();
                } catch (IOException e) {
                    System.err.println("Erro ao fechar socket: " +
                            e.getMessage());
                }
            }
        }
    }

    public static void main(String[] args) {
        new ImplCliente("localhost", 12345);
```

```
    }
}
```

4. Cliente/servidor.
   - Fluxo:
     - Cliente digita qualquer mensagem ("oi", "mensagem bem maior...", etc.).
     - Servidor lê e responde o que o operador do servidor digitar no console.
     - Tanto cliente quanto servidor podem enviar mensagens de qualquer tamanho (até o limite do buffer).
     - Se o cliente digitar "sair", a comunicação fechará.

Java

```java
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Scanner;

public class ImplServidor {

    private int porta;

    public ImplServidor(int porta) {
        this.porta = porta;
        rodar();
    }

    private void rodar() {
        ServerSocket servidor = null;
        try {
            servidor = new ServerSocket(porta);
            System.out.println("Servidor aguardando conexões na porta " +
                      porta + "...");

            while (true) {
                Socket cliente = servidor.accept();
                System.out.println("Cliente conectado: " +
                             cliente.getInetAddress());

                try {
                    InputStream entrada = cliente.getInputStream();
                    OutputStream saida = cliente.getOutputStream();
                    Scanner teclado = new Scanner(System.in);

                    byte[] buffer = new byte[1024];
                    int bytesLidos;

                    while ((bytesLidos = entrada.read(buffer)) != -1) {
                        String msgRecebida =
                                new String(buffer, 0, bytesLidos, "UTF-8");
                        System.out.println("Mensagem recebida: " + msgRecebida);

                        if (msgRecebida.equalsIgnoreCase("sair")) {
                            System.out.println("Cliente encerrou a comunicação.");
                            break;
                        }

                        System.out.print("Digite resposta: ");
                        String resposta = teclado.nextLine();
```

```
                    saida.write(resposta.getBytes("UTF-8"));
                    System.out.println("Resposta enviada: " + resposta);
                }

                entrada.close();
                saida.close();
                cliente.close();

            } catch (IOException e) {
                System.err.println("Erro na comunicação com cliente: " +
                        e.getMessage());
            }
        }

    } catch (IOException e) {
        System.err.println("Erro no servidor: " + e.getMessage());
    } finally {
        if (servidor != null) {
            try {
                servidor.close();
            } catch (IOException e) {
                System.err.println("Erro ao fechar servidor: " +
                        e.getMessage());
            }
        }
    }
}

public static void main(String[] args) {
    new ImplServidor(12345);
}
}
```

Java

```
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;
import java.util.Scanner;

public class ImplCliente {

    private String host;
    private int porta;

    public ImplCliente(String host, int porta) {
        this.host = host;
        this.porta = porta;
        rodar();
    }

    private void rodar() {
        Socket socket = null;
        try {
            socket = new Socket(host, porta);
            System.out.println("Conectado ao servidor!");

            OutputStream saida = socket.getOutputStream();
            InputStream entrada = socket.getInputStream();
```

```
            Scanner teclado = new Scanner(System.in);

            byte[] buffer = new byte[1024];
            int bytesLidos;

            while (true) {
                System.out.print("Digite mensagem: ");
                String mensagem = teclado.nextLine();
                saida.write(mensagem.getBytes("UTF-8"));

                if (mensagem.equalsIgnoreCase("sair")) {
                    break;
                }

                bytesLidos = entrada.read(buffer);
                if (bytesLidos == -1) {
                    System.out.println("Servidor encerrou a conexão.");
                    break;
                }

                String resposta =
                        new String(buffer, 0, bytesLidos, "UTF-8");
                System.out.println("Resposta recebida: " + resposta);
            }

            entrada.close();
            saida.close();

        } catch (IOException e) {
            System.err.println("Erro no cliente: " + e.getMessage());
        } finally {
            if (socket != null) {
                try {
                    socket.close();
                } catch (IOException e) {
                    System.err.println("Erro ao fechar socket: " +
                              e.getMessage());
                }
            }
        }
    }

    public static void main(String[] args) {
        new ImplCliente("localhost", 12345);
    }
}
```

5. Exercícios propostos.

1. Codifique os exemplos e observe o funcionamento de cada um. O objetivo é compreender quais passos são necessários para fazer a comunicação entre processos usando *sockets* em Java.
2. Pesquise sobre as classes `OutputStream` e `InputStream`. O objetivo é compreender a função dessas classes bem como os métodos utilizados nos exemplos (`read()` e `write()`).
3. Considere a necessidade de implementação de uma aplicação cliente/servidor, onde o servidor implementa as quatro operações matemáticas básicas (soma, subtração, multiplicação e divisão). Desenvolva um cliente, que faz chamadas a métodos no servidor para cada um das operações e mostra o resultado. Pense em uma boa forma de enviar e receber os parâmetros necessários (que você vai determinar quantos e quais) para que o servidor entenda qual operação realizar por vez.

6. Tamanho do *buffer*.

- Cada Socket em Java tem dois *buffers* principais:

    - *Receive Buffer.*
        - getReceiveBufferSize() / setReceiveBufferSize(int): Armazena os bytes recebidos da rede até que a aplicação os leia com *InputStream*.
    - *Send Buffer.*
        - getSendBufferSize() / setSendBufferSize(int): Armazena os bytes que a aplicação envia até que o sistema operacional consiga transmiti-los pela rede.
- Tamanho padrão.
    - O tamanho padrão não é fixo no Java, pois depende da configuração do sistema operacional (socket do SO), mas normalmente é:
        - Linux (atual):
            - SO_RCVBUF (recepção): 87 380 bytes (≈ 85 KB).
            - SO_SNDBUF (envio): 16 384 bytes (≈ 16 KB).
        - Windows:
            - SO_RCVBUF (recepção): 64 KB a 128 KB.
            - SO_SNDBUF (envio): 64 KB a 128 KB.
        - macOS:
            - SO_RCVBUF (recepção): 64 KB.
            - SO_SNDBUF (envio): 64 KB.

- Verificar na execução.

```java
import java.net.*;

public class TesteBuffer {
    public static void main(String[] args) throws Exception {
        Socket socket = new Socket();
        socket.connect(new InetSocketAddress("localhost", 12345));
        // conectar com um servidor ativo nessa porta

        System.out.println("Tamanho do buffer de recepção: " +
                socket.getReceiveBufferSize());
        System.out.println("Tamanho do buffer de envio: " +
                socket.getSendBufferSize());

        socket.close();
    }
}
```

- Alterar antes da conexão.

```java
Socket socket = new Socket();
socket.setReceiveBufferSize(131072); // 128 KB
socket.setSendBufferSize(131072);
socket.connect(new InetSocketAddress("localhost", 5000));

System.out.println("Novo buffer de recepção: " + socket.getReceiveBufferSize());
System.out.println("Novo buffer de envio: " + socket.getSendBufferSize());
```

7. Interfaces de rede e endereços.

- Objetivos do exemplo:
    - Exibir todas as interfaces de rede do sistema (nome, endereço IP, MAC, etc.);
    - Demonstrar o uso das classes *InetSocketAddress* e *InetAddress*, incluindo criação, extração de informações e resolução de endereços;
    - Mostrar como obter informações do host local e de um host remoto (ex: google.com).

Java

```java
import java.io.IOException;
import java.net.*;
import java.util.Enumeration;

public class InfoRede {
    public static void main(String[] args) {
        try {

            InetAddress localHost = InetAddress.getLocalHost();
            System.out.println("Hostname local: " + localHost.getHostName());
            System.out.println("Endereço IP local: " +
                      localHost.getHostAddress());
            System.out.println();

            // Obtendo o endereço de um site remoto
            InetAddress google = InetAddress.getByName("www.google.com");
            System.out.println("Nome canônico: " + google.getCanonicalHostName());
            System.out.println("Endereço IP do Google: " +
                      google.getHostAddress());
            System.out.println();

            InetSocketAddress enderecoServidor =
                    new InetSocketAddress("localhost", 5000);
            System.out.println("Endereço completo: " + enderecoServidor);
            System.out.println("HostName: " + enderecoServidor.getHostName());
            System.out.println("Porta: " + enderecoServidor.getPort());
            System.out.println("IP: " +
                      enderecoServidor.getAddress().getHostAddress());
            System.out.println("É resolvido? " +
                      !enderecoServidor.isUnresolved());
            System.out.println();

            System.out.println("=== Interfaces de rede disponíveis ===");

            // Lista todas as interfaces de rede do sistema
            Enumeration<NetworkInterface> interfaces =
                      NetworkInterface.getNetworkInterfaces();

            while (interfaces.hasMoreElements()) {
                NetworkInterface ni = interfaces.nextElement();
                System.out.println("Nome da interface: " + ni.getName());
                System.out.println("Display name: " + ni.getDisplayName());
                System.out.println("É ativa? " + ni.isUp());
                System.out.println("É loopback? " + ni.isLoopback());
                System.out.println("Suporta multicast? " +
                       ni.supportsMulticast());

                System.out.println();
            }

            System.out.println("=== Testando conexão com InetSocketAddress ===");
            InetSocketAddress destino =
                    new InetSocketAddress("example.com", 80);
            try (Socket socket = new Socket()) {
                socket.connect(destino, 2000); // timeout de 2 segundos
                System.out.println("Conectado a: " + destino);
                System.out.println("Endereço local usado: " +
                       socket.getLocalSocketAddress());
            } catch (IOException e) {
                System.out.println("Não foi possível conectar a: " + destino);
            }

            System.out.println("\n=== Criando ServerSocket com InetSocketAddress ===");
```

```
            InetSocketAddress localBind =
                    new InetSocketAddress("localhost", 6000);
            try (ServerSocket servidor = new ServerSocket()) {
                servidor.bind(localBind); // associa o servidor ao endereço e porta
                System.out.println("Servidor ouvindo em: " +
                       servidor.getLocalSocketAddress());
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Referências.

ORACLE. Class InetAddress (Java Platform SE 21). Oracle Documentation, 2024. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/InetAddress.html. Acesso em: 9 out. 2025.

ORACLE. Class InetSocketAddress (Java Platform SE 21). Oracle Documentation, 2024. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/InetSocketAddress.html. Acesso em: 9 out. 2025.

ORACLE. Class NetworkInterface (Java Platform SE 21). Oracle Documentation, 2024. Disponível em: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/NetworkInterface.html. Acesso em: 9 out. 2025.